

L2 总结:手写 agent loop

日期:2026-07-14 结果:达标  (作业跑通、quiz 5/5)

这节课学到了什么(一句话)

从零手写出一个**能用的 agent loop**:让模型「先用工具、拿到真实结果、再回答」,并自动重复多轮直到问题解决。落地为一个查订单迷你客服 agent。

核心内容

1. 为什么 L1 还不算 agent

L1 的 `hello_llm.py` 只有「LLM」,缺工具/记忆/循环。需要查订单系统时只能编。Agent loop 补上后,模型才能真正「去查再答」。

2. function calling 「四步握手」(本节最重要的 20%)

- ① 你→模型: 问题 + 工具说明书
- ② 模型→你: 不直接答,"点单"(tool_calls):要调 `query_order(order_id='A123')`
- ③ 你(本地): 真的执行 `query_order("A123")` → 得到结果
- ④ 你→模型: 结果塞回 `messages` → 模型给最终答复

命脉分工:模型决策(点哪个单),你的代码行动(真正执行)。模型不会自己连数据库。

3. agent loop 骨架

```
while True:
    发 messages(+tools)给模型
    if not msg.tool_calls:          # 没点单 → 拿到最终答案
        return msg.content        # ← 停止条件就在这
    messages.append(msg)          # 先 append 模型的点单消息
    for tool_call in msg.tool_calls:
        result = AVAILABLE_TOOLS[name](**args)  # 本地执行
        messages.append({"role":"tool", "tool_call_id":tool_call.id, "content":result})
    # 带着工具结果再问一轮
```

课堂上真实发生的事(这节的高光)

作业一次跑通,还无意中验证了三个点

学习者补完 loop 后跑 `order_agent.py`,一次运行同时覆盖了: - **A123 + B456 两个订单都查出来** → 一轮里两次点单, for 循环逐个执行、都塞回,没漏。 - 「**几点下班**」没调工具、**直接答** → 模型自己判断出这问题不需要工具(靠 `description` 决策)。 - **没死循环、没报 tool 对不上** → 停止条件与 append 顺序都对。

学习者问出的三个好问题(收进讲义)

1. **TOOLS schema 是什么** → 它是给模型看的「菜单」,本身不执行; `name` 是和真函数对上号的线; `description` 是决定「调不调」的指令(面试考点:打磨 `description` 提升工具调用准确率); `parameters` 是 JSON Schema,参数名必须和真函数签名一致。
2. **怎么在环境里设 API key**(延续 L1 环境短板)→ PowerShell `$env:X="..."` / `setx`; macOS zsh `export`; 铁律:设 key 与跑代码必须同一个终端会话。
3. **LLM 判断结束后连接会不会断、上下文会不会丢、用户追问会不会忘**(见下方心智模型)。

本节最该带走的心智模型

你和 LLM 之间没有「持续连接」。每次 `create(...)` 都是无状态的一次性请求,模型天生不记得上一轮——它「看起来记得」只因为你每轮重发了整个 `messages`。记忆不在模型,在你手里的 `messages` 数组。→ 当前 `run_agent` 每次从头 new messages、return 就丢掉,所以用户追问会「忘光」。这正是 **L5 记忆与状态** 要解决的,学习者靠直觉提前推到了这里。

Quiz 结果:5/5

Q1 循环 vs 一次 ☒ / Q2 模型不连库、agent 本地执行 ☒ / Q3 删点单消息 → `tool_call_id` 对不上报错 ☒ / Q4 停止条件在 `return`、生产加循环次数上限兜底 ☒ (并追问出 L5 记忆问题)/ Q5 数组变长 → 上下文窗口上限、成本延迟上涨、lost in the middle → L6 ☒

掌握等级

Beginner → 稳固,局部触到 **Intermediate** 的思考。能独立手写 agent loop,并主动推导出记忆(L5)、上下文(L6)两个后续核心问题。工程基本功仍是需持续留意的短板(环境变量/终端会话)。

埋下的伏笔(后续呼应)

- `messages` 会越滚越长 → **L6 上下文长度管理**(学习者最关心的求职痛点)。
- 无状态 API、记忆要自己维护、追问会忘 → **L5 记忆与状态**。
- `description` 决定工具调用准确率、工具会越加越多 → **L3 tool use 深入** / **L4 多工具编排**。